

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO4 ASSESSMENT TOOL 2 — CI/CD PIPELINE DESIGN EXERCISE

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	G. Ram Charan	Reg. No.:	192525043
Date:		Duration:	60 minutes

Code of Conduct

I ___ G. Ram Charan (192525043) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___ G. Ram Charan (192525043) _____

Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System

CSA1017-SOFTWARE ENGINEERING

Name: - G. Ram Charan

Reg no: -192525043

1. Problem Statement Overview

1.1 Industry Context

Agriculture is increasingly adopting AI technologies to improve crop productivity and minimize losses caused by diseases and pests. Computer vision, drones, IoT sensors, and weather analytics help farmers monitor crop conditions and implement precision farming practices. Delivering these capabilities reliably to farmers requires a disciplined software delivery process, which is the focus of this exercise.

1.2 Problem Statement

Crop diseases significantly reduce agricultural productivity and often spread before farmers can detect them through manual inspection. The Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System analyzes crop images, monitors environmental conditions, predicts disease outbreaks, estimates crop yield, and recommends suitable treatments. Because farmers depend on this system during active growing seasons, new releases must reach production quickly, safely, and with the ability to roll back instantly if a defect is detected.

1.3 Objectives of This Exercise

This exercise analyzes the project requirements of the assigned system and identifies the CI/CD needs that follow from them, designs a complete pipeline workflow from source code management through build, test, code quality analysis, packaging, and deployment, selects and justifies suitable tools for every stage, defines automated testing strategies, designs a deployment strategy covering development, staging, and production environments, and incorporates security, quality checks, notifications, and rollback mechanisms throughout.

1.4 Purpose of This Document

This document is the CI/CD Pipeline Design deliverable for the assigned problem statement. It presents the pipeline architecture and diagram, the tools and configuration selected for each stage, the automated testing and quality strategy, the deployment, security and rollback strategy, and an explanation of the workflow from code commit to production deployment.

2. Requirement & Pipeline Analysis

2.1 Project Characteristics Relevant to CI/CD

The system is composed of several independently deployable components: a farmer-facing mobile application, a web analytics dashboard for enterprise farm managers, a set of backend microservices (disease detection, yield prediction, treatment recommendation, IoT and weather analytics), and a machine learning inference layer that is retrained and redeployed periodically. This multi-service, ML-dependent structure directly shapes the CI/CD needs identified below.

2.2 CI/CD Needs Identified

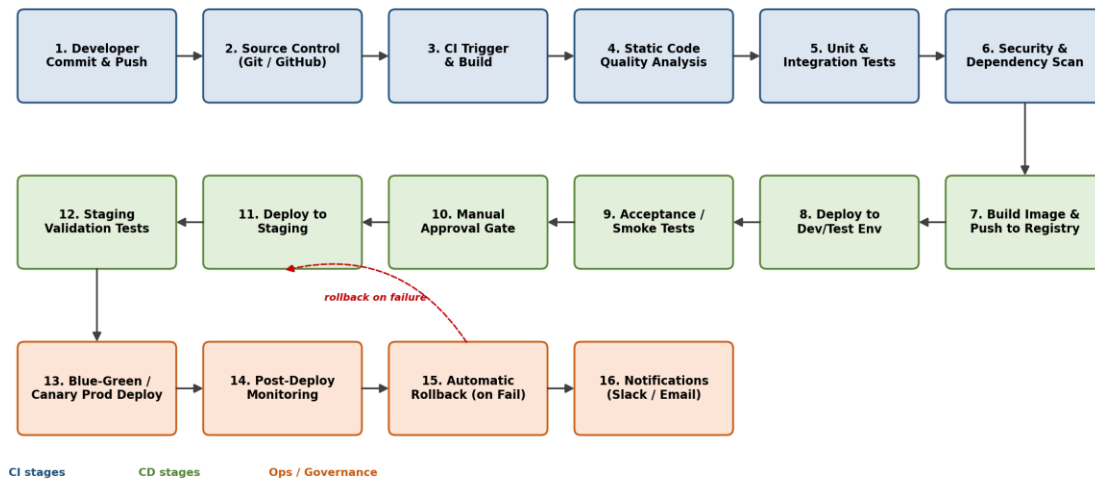
- Independent build and release pipelines per microservice, so that a change to the yield-prediction service does not force a redeploy of the disease-detection service.
- A dedicated validation step for newly trained machine learning models, distinct from ordinary unit testing, before a model artifact is allowed into the packaging stage.
- Automated code quality and security gates on every pull request, since the system processes farmer-submitted images and sensor data and therefore has a meaningful attack surface.
- A safe, low-risk deployment strategy for production, because farmers rely on the outbreak-alert and diagnosis features during active growing seasons and cannot tolerate extended downtime.
- Fast, reliable rollback, since a defective release of the outbreak-prediction logic could cause farmers to miss a genuine disease-risk warning.
- Environment parity between development, staging, and production, so that issues caused by IoT sensor integration or third-party weather APIs are caught before reaching farmers.
- Notification and audit trails for every deployment and rollback event, to support the DevOps engineers and agronomists who are accountable for system reliability.

2.3 Scope of the Pipeline

The pipeline designed in this document covers the complete journey from a developer's code commit to a production release that is live and being monitored, including the automated safety net that triggers a rollback if the release proves unhealthy.

3. Pipeline Architecture & Workflow

The diagram below shows the complete, sixteen-stage CI/CD pipeline designed for this system. The top row (blue) covers Continuous Integration, the middle row (green) covers pre-production delivery, and the bottom row (orange) covers production release, monitoring, and rollback governance.



3.1 Continuous Integration (CI) Stages

- Stage 1. Developer Commit & Push — a developer commits code for one microservice to a feature branch and pushes it to the remote repository.
- Stage 2. Source Control (Git/GitHub) — the push opens or updates a pull request against the main branch, which is protected and requires at least one peer review.
- Stage 3. CI Trigger & Build — the pull request automatically triggers the CI workflow, which installs dependencies and compiles/builds the affected service.
- Stage 4. Static Code Quality Analysis — the build artifact is analyzed for code smells, duplication, and maintainability issues, with the pipeline failing the check if the quality gate is not met.
- Stage 5. Unit & Integration Tests — automated unit tests validate individual functions, and integration tests validate interaction between the changed service and its immediate dependencies (database, message queue, or adjacent service).
- Stage 6. Security & Dependency Scanning — the code and its third-party dependencies are scanned for known vulnerabilities before the build is allowed to proceed.

3.2 Delivery (CD) Stages

- Stage 7. Build Image & Push to Registry — a Docker image is built for the service and pushed to a private container registry, tagged with the commit hash.
- Stage 8. Deploy to Dev/Test Environment — the new image is automatically deployed to an ephemeral development namespace for further validation.

- Stage 9. Acceptance / Smoke Tests — a lightweight suite of end-to-end tests confirms that core user journeys (image upload and diagnosis, sensor data ingestion, yield forecast request) work against the freshly deployed build.
- Stage 10. Manual Approval Gate — a designated reviewer (DevOps lead or agronomist for model changes) approves promotion of the build to staging.
- Stage 11. Deploy to Staging — the approved image is deployed to a staging environment that mirrors production configuration, including connected (non-production) IoT and weather data feeds.
- Stage 12. Staging Validation Tests — a fuller regression suite, including ML model accuracy checks, runs against staging before production promotion is permitted.

3.3 Production Release & Governance Stages

- Stage 13. Blue-Green / Canary Production Deployment — the release is rolled out to a small percentage of production traffic (canary) or to a parallel environment that is swapped in (blue-green), limiting the blast radius of any defect.
- Stage 14. Post-Deployment Monitoring — key health indicators (inference latency, error rate, alert-delivery success rate) are watched automatically for a defined observation window.
- Stage 15. Automatic Rollback (on Failure) — if monitored thresholds are breached during the observation window, the pipeline automatically reverts traffic to the last known-good release.
- Stage 16. Notifications — the outcome of every stage, and in particular deployment success, deployment failure, and rollback events, is posted to the DevOps team's Slack channel and emailed to the release owner.

4. Tools & Technology Selection

The single consolidated table below lists the tool or technology selected for every stage of the pipeline described in Section 3, together with the justification for that selection based on the needs identified in Section 2.

Pipeline Stage	Tool / Technology Selected	Justification
Source Code Management	Git with GitHub (private repository, branch protection on main)	Industry-standard distributed VCS; branch protection and required pull-request reviews prevent unreviewed code reaching the pipeline.
CI Orchestration	GitHub Actions	Tightly integrated with the source repository, no separate CI server to maintain, and scales well for a small-to-mid engineering team.

Pipeline Stage	Tool / Technology Selected	Justification
Build / Dependency Management	Docker multi-stage builds; pip/npm/Maven per service	Multi-stage builds keep final images small and reproducible across the mobile backend, web dashboard, and ML services.
Static Code Quality Analysis	SonarQube (or SonarCloud)	Detects code smells, duplication, and maintainability issues automatically on every pull request, enforcing a quality gate before merge.
Unit & Integration Testing	PyTest (ML/backend services), JUnit (Java modules), Jest (web dashboard)	Matches the language stack of each service; produces machine-readable coverage reports the pipeline can gate on.
ML Model Validation	MLflow + custom accuracy/regression threshold checks	Confirms that a newly trained CNN/yield model meets a minimum accuracy bar before it is packaged for deployment, preventing model regression.
Security & Dependency Scanning	Trivy (container image scan), OWASP Dependency-Check, GitHub Dependabot	Combined scanning of the Docker image, third-party libraries, and known CVEs closes the most common supply-chain and image vulnerabilities.
Secrets Management	HashiCorp Vault / GitHub Actions Encrypted Secrets	Keeps API keys (weather provider, SMS gateway) and database credentials out of source control and pipeline logs.
Artifact Packaging & Registry	Docker, Amazon ECR (or Docker Hub private repository)	Provides versioned, immutable container images that can be promoted unchanged from staging to production.
Infrastructure as Code	Terraform	Ensures the Dev, Staging, and Production environments are provisioned consistently and changes are peer-reviewed before applying.
Deployment Orchestration	Kubernetes (managed, e.g. Amazon EKS)	Supports auto-scaling of the inference and IoT-ingestion services and enables blue-green/canary rollout strategies natively.
Deployment Automation	Argo CD (GitOps-based continuous delivery)	Declarative, auditable deployments driven from a Git manifest, simplifying rollback to any previous known-good revision.
Monitoring & Alerting	Prometheus + Grafana, with Alertmanager	Tracks inference latency, error rates, and infrastructure health; feeds the automatic rollback and notification stages.

Pipeline Stage	Tool / Technology Selected	Justification
Notifications	Slack webhook integration, email via SMTP/SES	Immediately informs the DevOps and agronomist teams of pipeline failures, successful releases, and rollback events.
Centralized Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	Aggregates logs from all microservices for fast root-cause analysis when a deployment or rollback event occurs.

5. Automated Testing & Quality Integration

Automated testing and quality checks are embedded directly into the pipeline rather than run manually after the fact, so that a defect is caught as early as possible and as close as possible to the stage that introduced it.

5.1 Unit Testing

Every microservice carries its own unit test suite (PyTest for the Python-based ML and backend services, JUnit for any Java components, Jest for the web dashboard front end) which runs on every commit. The pipeline enforces a minimum code coverage threshold of 80% for new code and fails the build if the threshold is not met.

5.2 Integration Testing

Integration tests exercise the interaction between a service and its immediate dependencies, for example confirming that the Disease Detection Service correctly calls the AI/ML Inference Engine and persists the result to the Image and Diagnosis Data Store. These tests run against containerized dependencies spun up specifically for the CI job, keeping the test environment isolated and repeatable.

5.3 Machine Learning Model Validation

Because the system depends on a CNN vision model and a yield regression model, the pipeline includes a dedicated validation stage that compares a newly trained model's accuracy, precision, and recall against a held-out validation dataset and against the currently deployed model. A new model is only packaged for deployment if it meets or exceeds the minimum accuracy threshold and does not regress against the previous version.

5.4 Code Quality and Static Analysis

Static analysis (via SonarQube) checks every pull request for code smells, duplicated logic, cyclomatic complexity, and maintainability issues, and blocks merges that fall below the configured quality gate. This keeps the codebase maintainable as more contributors add features across the disease detection, yield prediction, and analytics modules.

5.5 Security Testing

Security testing combines static application security testing (SAST) on the source code, software composition analysis on third-party dependencies to catch known CVEs, and container image scanning immediately after the Docker image is built. Any critical or high-severity finding fails the pipeline and blocks promotion until it is resolved or explicitly risk-accepted by a security reviewer.

5.6 Acceptance and Regression Testing

A smoke-test suite validates the core farmer journeys (image upload and diagnosis, sensor registration, yield forecast, treatment lookup) immediately after each deployment to Dev/Test and Staging. A broader regression suite runs in Staging before production promotion, covering boundary and negative scenarios similar to those defined in the corresponding Test Case Design document for this problem statement.

6. Deployment, Security & Rollback Strategy

6.1 Environment Strategy

Four environments are maintained: Development (ephemeral, recreated per feature branch for early integration testing), Test (shared, stable environment for the QA/acceptance suite), Staging (a full mirror of production configuration, including non-production IoT and weather integrations, used for final validation), and Production (serving live farmers, farm managers, and agronomists). Configuration differences between environments are managed entirely through environment variables and Infrastructure as Code, never through manual changes, so that what is validated in Staging is what actually reaches Production.

6.2 Deployment Strategy

Production releases use a canary deployment strategy: the new version initially receives a small percentage of live traffic (for example 10%) while post-deployment monitoring observes error rates and inference latency. If the canary remains healthy for the defined observation window, traffic is progressively shifted to 100%. For releases that touch the core diagnosis or outbreak-alert logic, a blue-green deployment is used instead, allowing an instantaneous full cutover and an equally instantaneous cutover back if a problem is detected.

6.3 Security Practices Embedded in the Pipeline

- Branch protection and mandatory code review on the main branch prevent unreviewed code from reaching any deployable environment.
- Secrets (weather API keys, SMS gateway credentials, database passwords) are stored in a dedicated secrets manager and injected at deploy time, never committed to source control.
- Container images are scanned for vulnerabilities before being pushed to the registry, and only signed, scanned images can be deployed to Staging or Production.
- Role-based access control restricts who can approve the Manual Approval Gate and who can trigger a production deployment or rollback.

- All production traffic, including farmer image uploads, is encrypted in transit using TLS, and stored diagnosis images and sensor data are encrypted at rest.

6.4 Quality Checks and Notifications

Every pipeline stage reports its pass/fail status back to the pull request or deployment record, so the current health of a release is always visible without needing to inspect logs manually. Deployment starts, successful completions, failures, and rollback events are all posted automatically to a dedicated Slack channel and emailed to the release owner, ensuring the DevOps team and relevant agronomists are informed in near real time.

6.5 Rollback Mechanism

Rollback is automatic and does not depend on a human noticing a problem first. Post-deployment monitoring continuously compares live error rate, latency, and (for model releases) prediction-confidence distribution against defined thresholds during the observation window. A threshold breach automatically triggers Argo CD to redeploy the last known-good Git-tracked revision, restoring service within minutes. A manual rollback command is also available to the on-call DevOps engineer for issues that automated monitoring does not directly measure, such as a farmer-reported incorrect treatment recommendation.

7. Pipeline Diagram & Documentation

This section documents the end-to-end workflow shown in the diagram in Section 3, walking through what happens from the moment a developer commits code to the moment that change is live in production and being watched by the automated rollback safety net.

7.1 Workflow Walkthrough: Commit to Deployment

A developer working on, for example, an improvement to the outbreak-risk detection logic commits their change to a feature branch and opens a pull request. This immediately triggers the CI workflow, which builds the affected service, runs static code quality analysis, executes the unit and integration test suites, and performs security and dependency scanning, all before a human reviewer even looks at the code. Only once every automated check passes and a peer reviewer approves the pull request is the change merged into the main branch.

The merge triggers the delivery pipeline: a Docker image is built and pushed to the container registry, then automatically deployed to the Dev/Test environment where smoke tests confirm the core farmer journeys still work. A designated reviewer then approves promotion to Staging, where the fuller regression suite, including machine learning model validation, runs against a production-like environment with realistic IoT and weather integrations.

Once Staging validation passes, the release is promoted to Production using a canary or blue-green strategy depending on the risk profile of the change. Post-deployment monitoring watches the release for a defined observation window; if the release remains healthy, it is promoted to full traffic and the team is notified of a successful deployment. If monitored thresholds are breached at any point, the pipeline

automatically rolls back to the previous known-good release and immediately notifies the DevOps team, closing the loop without requiring manual intervention to protect farmers from a defective release.

7.2 Documentation Notes

The diagram in Section 3 is colour-coded to distinguish Continuous Integration stages (blue), pre-production delivery stages (green), and production release and governance stages (orange), with a dashed red arrow indicating the automatic rollback path. All sixteen stages are numbered in execution order, and the tools supporting each stage are documented in the single Tools & Technology table in Section 4, so that the architecture, the workflow, and the technology choices can all be cross-referenced from one consistent numbering scheme.

8. Coverage Summary

This design addresses Requirement and Pipeline Analysis by deriving the CI/CD needs directly from the multi-service, ML-dependent nature of the assigned system (Section 2). It addresses Pipeline Architecture and Workflow with a complete, sixteen-stage pipeline spanning source control through build, quality, testing, packaging, and deployment (Section 3). Tools and Technology Selection is covered by the single consolidated table mapping a justified tool to every stage (Section 4). Automated Testing and Quality Integration is addressed through the layered testing strategy covering unit, integration, ML model validation, static analysis, and security testing (Section 5). Deployment, Security and Rollback Strategy is addressed through the four-environment model, canary/blue-green deployment, embedded security practices, and automatic rollback mechanism (Section 6). Finally, the Pipeline Diagram and accompanying commit-to-deployment walkthrough (Section 3 and Section 7) provide the clear, documented diagram required by the assignment.

9. Conclusion

This document has analyzed the CI/CD requirements of the Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System, designed a complete sixteen-stage pipeline covering Continuous Integration and Continuous Deployment, selected and justified appropriate tools for every stage, defined a layered automated testing and quality strategy including dedicated machine learning model validation, and designed a deployment strategy with canary/blue-green releases, embedded security practices, notifications, and automatic rollback. Together, the architecture diagram, tools table, and workflow documentation in this report fulfil the CI/CD Pipeline Design deliverable requested in the assignment instructions.